

Guia de Pipelines

SICAD - Sistema de Control de Acceso Digital

Que es una pipeline?

Una **pipeline** es una serie de pasos **automaticos** que GitHub ejecuta por si solo cuando ocurre algo en el repositorio (subes codigo, marcas una version, apruebas cambios a main). Se configuran con GitHub Actions, en archivos dentro de la carpeta **.github/workflows/**. SICAD tiene **tres** pipelines:

Pipeline	Archivo	Cuando se ejecuta
CI - Build & Test	ci.yml	Push o PR a dev, qa o main
Release	release.yml	Al crear un tag de version (v1.0.0)
CD - Deploy a Azure	deploy.yml	Al mergear a main

1. CI - Build & Test (ci.yml)

Archivo:

.github/workflows/ci.yml

Cuando se ejecuta:

Cada vez que alguien sube código (push) o abre un Pull Request hacia las ramas dev, qa o main.

Que hace (paso a paso):

1. Descarga el código del repositorio.
2. Instala Node.js y las dependencias del proyecto.
3. Backend: genera el cliente de Prisma y corre las pruebas con Jest.
4. Frontend: revisa el estilo del código (lint) y lo compila (build).

Para que sirve:

Vigila que cada cambio no rompa nada **antes** de mezclarlo. Si algo falla, la pipeline se marca en rojo y el Pull Request no debe aceptarse. Es el filtro de calidad automático del equipo.

2. Release (release.yml)

Archivo:

.github/workflows/release.yml

Cuando se ejecuta:

Solo cuando se crea y sube un tag de versión, por ejemplo: git tag v1.0.0 && git push --tags.

Que hace (paso a paso):

1. Construye las imágenes Docker del backend y del frontend.
2. Publica un Release oficial en GitHub con notas de la versión generadas automáticamente.

Para que sirve:

Sirve para **empaquetar y marcar una versión estable** del proyecto, dejando un punto de referencia claro (v1.0.0, v1.1.0, etc.) de lo que se entregó.

3. CD - Deploy a Azure (deploy.yml)

Archivo:

.github/workflows/deploy.yml

Cuando se ejecuta:

Cada vez que algo llega a la rama main (es decir, cuando QA aprueba cambios y se mergean a main).

Que hace (paso a paso):

1. Construye las imágenes Docker del backend y del frontend.
2. Las publica en el registro de contenedores (ghcr.io).
3. Inicia sesión en Azure con las credenciales guardadas como secreto.
4. Despliega las imágenes en Azure App Service: una para el backend y otra para el frontend.

Para que sirve:

Hace el **despliegue continuo**: la aplicación en producción se actualiza sola al aprobar cambios a main. En resumen: **mergeas a main y se sube solo a Azure**, dockerizada.

Requisito: haber creado los recursos en Azure y guardado el secreto AZURE_CREDENTIALS en GitHub.

Como encajan las tres (flujo general)

El trabajo avanza por las ramas y cada pipeline entra en su momento:

Rama / evento	Pipeline que corre
feature/* → dev	CI (build + pruebas)
dev → qa	CI (build + pruebas)
qa → main (aprobado por QA)	CI + CD (deploy a Azure)
crear tag v* (version)	Release (imagenes + release en GitHub)

En pocas palabras: **CI** revisa cada cambio, **Release** marca versiones estables y **CD** publica en Azure lo aprobado a main. Juntas cubren los tres automatismos: build, release y despliegue.